# Speculative Speculative Decoding: Hiding Draft Latency Through Asynchronous Speculation on Multi-GPU Systems

**Marcus Alenius** **William Chien**
Carnegie Mellon University
{malenius, wjchien}@andrew.cmu.edu

**URL:** https://alenius.io/15418-project

## 1 Summary

Speculative decoding (SD) accelerates large language model inference by having a small draft model propose tokens that a larger target verifies in parallel, but draft and verification still alternate sequentially. Speculative speculative decoding (SSD) eliminates this serialization by pre-speculating for likely verification outcomes on a dedicated draft GPU concurrently with target verification. We implement SD and SSD from scratch in C++/CUDA against cuBLAS and NCCL, evaluating them on a transformer-shaped compute graph matching Llama-3.2-1B and Llama-3.1-8B. On V100 nodes with 1D tensor parallelism, SD achieves $4.08\times$ over autoregressive decoding (within 3% of theory) and SSD reaches $5.67\times$, beating SD by 32–53% across acceptance rates $\alpha \in [0.5, 1.0]$. We find that SSD's optimal speculation length is set by the verification window rather than the SD speedup curve, that fan-out is effectively free up to $F{=}64$, and that SSD with 2-way tensor parallelism Pareto-dominates every other configuration on both throughput and latency.

## 2 Background

### 2.1 Autoregressive Decoding

Large language models generate text autoregressively, meaning that each token is predicted based on all previous tokens. This creates a sequential dependency: the model cannot start generating the next token until the current one is complete. Each inference step (generating one token) involves multiple vector-matrix multiplies, barely utilizing the GPU's compute capacity. Yet, the dependence between steps means the GPU cannot move on to the next step until the current one finishes. The result is a powerful GPU that is underutilized during language model inference.

### 2.2 Workload Characterization

The input to autoregressive decoding is a token sequence (the prompt) and the output is a continuation of $N$ tokens sampled from the model. The model itself is a stack of $L$ transformer layers, where each layer is dominated by seven matrix multiplications: four for the attention block (Q, K, V, and output projections) and three for the feed-forward block (gate, up, and down projections). Element-wise operations like LayerNorm, softmax, and activation functions contribute negligibly to runtime and we omit them throughout.

At decode time, each step generates one token, which means each of the seven matrix multiplications has the shape $(1 \times H) \times (H \times \cdot)$, that is a vector-matrix product. The arithmetic intensity of these GEMMs is roughly one FLOP per byte: each weight is read from HBM once and used in a single multiply-accumulate. This puts the workload firmly in the memory-bandwidth-bound regime of the roofline. Adding more compute does not help but adding more bandwidth does.

Within a single GEMM, parallelism is abundant and SIMD-friendly: every output element is an independent dot product, which maps cleanly onto GPU warps and is exactly what cuBLAS exploits. *Across* decode steps, however, parallelism is zero: token $N + 1$ depends on the output of token $N$. This is the dependency that speculative decoding and SSD attack.

Tensor parallelism partially addresses the bandwidth problem by sharding each weight matrix across $T$ GPUs, so each GPU reads only $1/T$ of the weights per step. The cost is an all-reduce after the attention block and another after the FFN block to recombine partial sums. For a single decode step, the messages are small ($M \times H$ FP16 values per all-reduce), and at low $M$ the all-reduce latency competes with the per-GPU compute time, limiting how far TP can scale. Speculative decod-

ing exploits this same effect at the algorithm level: by batching $K$ candidate tokens into a single forward pass, it raises $M$ from 1 to $K$ and converts a sequence of bandwidth-bound steps into one with higher arithmetic intensity.

### 2.3 Speculative Decoding

**Speculative decoding (SD)** (Leviathan et al., 2023; Chen et al., 2023) attacks the across-token dependency directly. A small, fast draft model generates $K$ candidate tokens autoregressively, where $K$ is a hyperparameter (the speculation length). The target model then verifies all $K$ tokens in a single forward pass with $M = K$, raising the arithmetic intensity of the round's most expensive operation. The target accepts the longest prefix of candidates that matches what it would have generated and samples one bonus token. When the acceptance rate is reasonably high, multiple tokens are generated per target forward pass, parallelizing the decoding process and improving latency. Crucially, this process is lossless—the output distribution is identical to what the target model would have produced on its own.

SD breaks the dependency across tokens but introduces a new one within a round: the draft must finish all $K$ steps before verification can start, and verification must finish before the next draft can start. Drafting and verification alternate but never overlap, so much of the GPU sits idle during drafting.

### 2.4 Speculative Speculative Decoding

**Speculative speculative decoding (SSD)** (Kumar et al., 2026) breaks the within-round draft-verify dependency by running the two phases concurrently on separate hardware. While the target verifies round $R$'s speculation, the draft (on a dedicated GPU) predicts what the verification outcome will be—both the number of accepted tokens $k \in [0, K]$ and the bonus token sampled by the target—and pre-computes speculations for $F$ candidate outcomes in parallel, where $F$ is a hyperparameter called the fan-out. Pre-speculation is data-parallel across outcomes: all $F$ branches share the same draft model and run as a single batched forward pass with $M = F$, which raises the draft's arithmetic intensity in the same way SD's verification raises the target's.

The pre-computed branches are stored in a speculation cache, a dictionary keyed by predicted verification outcome $(k, t^*)$ with values being the $K$-token draft sequence for that branch. When the actual outcome arrives from the target, the draft performs a cache lookup: a hit returns the pre-computed speculation immediately and the draft latency is fully hidden behind verification, while a miss falls back to generating a fresh speculation just-in-time, putting $K$ serial draft passes back on the critical path. The speculation is always verified by the target, so SSD is lossless for the same reason SD is. Figure 1 illustrates how SSD overlaps drafting and verification, comparing it to SD.

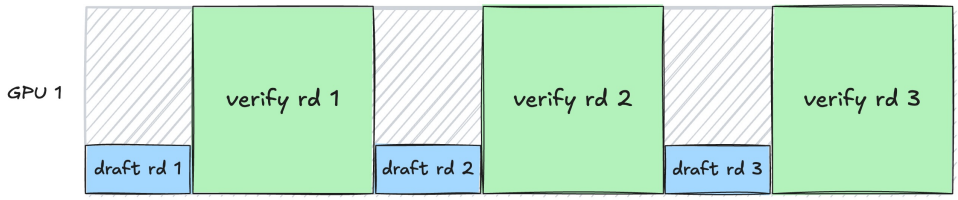
Two dependencies remain. First, the draft and target must exchange one round-trip per round—the target sends the verification outcome, the draft returns the speculation—which competes for interconnect bandwidth with the tensor-parallel all-reduces inside the target's forward pass. Second, on a cache miss the round's latency degrades to roughly that of SD, so SSD's advantage depends on the cache hit rate $p_{hit}$ being high.

## 3 Approach

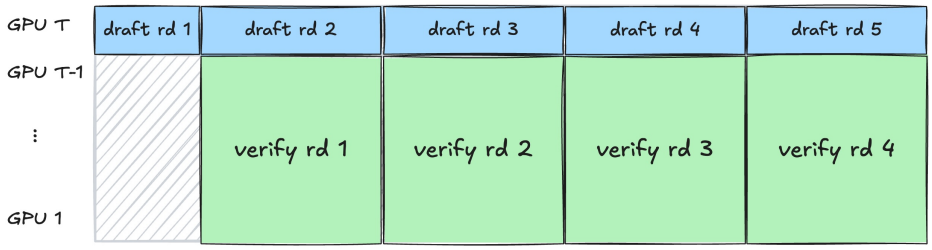
### 3.1 Platform and Technologies

**Languages and APIs.** We built the entire system from scratch in C++17 with CUDA, against NVIDIA's low-level GPU and multi-GPU APIs. The technology stack has four pieces:

- **cuBLAS** for all matrix multiplications. Every GEMM in the compute graph (Section 3.2) is a single `cublasHgemm` call in FP16, with tensor-core acceleration enabled via `CUBLAS_TENSOR_OP_MATH`.
- **NCCL** for inter-GPU communication. We use it in two distinct patterns: `ncclAllReduce` in `ncclSum` mode over FP16 for the tensor-parallel partial-sum recombination inside the target's forward pass (Section 3.3), and point-to-point `ncclSend`/`ncclRecv` for the outcome and speculation exchange between the target and draft GPUs in SSD (Section 3.5). The two patterns run on separate communicators so they do not serialize against each other.
- **CUDA streams and events.** Streams give us the asynchronous kernel scheduling we need to overlap independent work—most importantly, the SSD draft GPU runs pre-speculation GEMMs and outcome-receive on two separate streams so the GEMM launches and the NCCL recv can execute concurrently on the SMs and NICs respectively. CUDA events bracket the timed regions of every




(a) *SD on one GPU.* The draft model drafts $K$ tokens sequentially, then the target model verifies the $K$ tokens in parallel. Much of the GPU sits idle during the draft phase.




(b) *SSD on $T$ GPUs.* The draft model drafts $K$ tokens for multiple verification outcomes while the target model verifies the $K$ tokens from the previous round. All GPUs are busy.

Figure 1: Comparison of speculative decoding (SD) and speculative speculative decoding (SSD). SSD overlaps drafting and verification across GPUs, improving utilization.

benchmark.

- **C++ `std::thread`, `pthread_barrier_t`, and `std::atomic`.** We use one OS thread per GPU, since cuBLAS handles, NCCL communicators, and CUDA streams are all bound to a calling thread's current device. CUDA and NCCL primitives synchronize work on a stream but do not synchronize across host threads, so we use `pthread_barrier_t` to keep the $T$ target threads in lockstep across phases of an SD or SSD round, and a `std::atomic<bool>` for the end-of-run flag that target rank 0 writes and the others read.

**Hardware.** Development and single-GPU validation was performed on GHC cluster machines (NVIDIA GeForce RTX 2080, 8 GB). All benchmarking was run on PSC Bridges-2 V100-32GB nodes, each providing 8 NVIDIA Tesla V100 GPUs connected by NVLink 2.0 with ∼300 GB/s bidirectional bandwidth between GPU pairs.

**Why this platform.** Working in C++/CUDA against cuBLAS and NCCL directly—rather than through a higher-level framework like PyTorch or an inference engine like vLLM or TensorRT-LLM—gives us precise control over the four things that determine SSD's performance: (i) the order in which kernels are launched on each stream, (ii) the topology and rank assignment of every NCCL communicator, (iii) the placement of work across CUDA streams, and (iv) the placement of the draft loop relative to the target's forward pass across GPUs. A higher-level framework would hide these decisions behind its own scheduler. Since the contribution of SSD is exactly a coordination strategy across these four dimensions, working at the level of the underlying APIs is the natural choice.

### 3.2 Transformer-Shaped Compute Graph

Both SD and SSD are scheduling and coordination strategies layered on top of transformer decoding. Their performance depends on the relative latencies of draft and target forward passes, on how those forward passes overlap with inter-GPU communication, and on how tensor parallelism partitions work across devices. None of this depends on the numerical values produced by the model. Because we sample acceptance synthetically from a Bernoulli at a configurable rate $\alpha$ (Section 3.4), the durations and dependencies of the GEMMs are what determine our measurements, the actual numbers flowing through them are not.

We therefore evaluate our pipelines against a *transformer-shaped compute graph*—a stripped-down stand-in that preserves the wall-clock cost structure of a real decoder layer while removing everything irrelevant to the parallelism question.

This choice keeps the project focused on the systems contribution. Building a correct full-fidelity transformer in C++/CUDA from scratch (RM-

SNorm, rotary embeddings, fused attention with a paged KV cache, sampling, tokenization) would greatly expand the project scope on infrastructure that has no bearing on the performance of SSD relative to SD and AR. Using an existing inference engine such as vLLM or TensorRT-LLM would, in the opposite direction, hide exactly the things we want control over: kernel launches, NCCL collectives, stream synchronization, and the placement of the draft loop relative to the target's forward pass. A GEMM-only graph gives us full control over those primitives without committing to a model implementation.

**Why this is a faithful substitute.** At batch size 1 and modest sequence lengths, a transformer decode step is overwhelmingly dominated by the dense matrix multiplications in the attention projections and the feed-forward block. The remaining operations—RMSNorm, SiLU, the gate–up elementwise product, residual additions, and the softmax inside attention—are elementwise kernels whose cost is negligible relative to the projection GEMMs.[1] The operations that survive in our compute graph—the seven projection GEMMs listed below—are the same ones that dominate the timeline of a real Llama forward pass, and they exhibit the same arithmetic intensity, the same memory traffic, and the same dependency structure. Critically, the durations $T_{\text{draft}}$ and $T_{\text{target}}$ that drive the SD and SSD speedup formulas are determined almost entirely by these GEMMs, so the ratios our benchmark measures between AR, SD, and SSD round times track what one would observe on a production stack.

**Implementation.** We model each transformer layer as a sequence of seven half-precision GEMMs issued through cuBLAS kernel launches, mirroring a Llama-style decoder block with grouped-query attention and a SwiGLU feed-forward network. Table 1 lists the operations of one Llama decoder layer in execution order, with the seven GEMMs we model marked. Here $d_q = n_{\text{heads}} \cdot d_{\text{head}}$ and $d_{kv} = n_{kv\text{-heads}} \cdot d_{\text{head}}$ encode the GQA structure.

A full forward pass stacks $L$ such layers. Weights for each layer are allocated once at startup as FP16 buffers initialized via `cudaMemset` to `0x3C` (which encodes $\approx 1.0$ in FP16). Two scratch buffers, sized to $M \cdot \max(d_q, d_{\text{ff}})$, hold intermediate activations and are reused across layers, so the only per-layer state on the GPU is the weight matrices themselves.

We instantiate this graph for two configurations from the Llama family (Llama-3.2-1B and Llama-3.1-8B), specified by their published $(d_{\text{model}}, n_{\text{heads}}, n_{kv\text{-heads}}, d_{\text{head}}, d_{\text{ff}}, L)$ tuples (see Table 2). All experiments in Section 4 use these dimensions, so the GEMM shapes, weight footprints, and per-layer arithmetic intensity match the real models even though no real weights are ever loaded.

Table 1: Operations in one Llama-style decoder layer, in execution order, with shapes for an input of $M$ tokens and current sequence length $S$. The compute graph models the seven projection GEMMs. RMSNorms, RoPE, the KV cache append, attention, the SwiGLU elementwise step, and residual additions are omitted as discussed in the text.

| # | Operation | Shape |
|---|---|---|
| – | RMSNorm (pre-attention) | *skipped* |
| 1 | $Q = X\,W_Q$ | $(M, d) \times (d, d_q)$ |
| 2 | $K = X\,W_K$ | $(M, d) \times (d, d_{kv})$ |
| 3 | $V = X\,W_V$ | $(M, d) \times (d, d_{kv})$ |
| – | RoPE on $Q, K$ | *skipped* |
| – | Attention | *skipped* |
| 4 | $O = \text{attn}\,W_O$ | $(M, d_q) \times (d_q, d)$ |
| – | Residual add | *skipped* |
| – | RMSNorm (pre-FFN) | *skipped* |
| 5 | gate $= X\,W_{\text{gate}}$ | $(M, d) \times (d, d_{\text{ff}})$ |
| 6 | up $= X\,W_{\text{up}}$ | $(M, d) \times (d, d_{\text{ff}})$ |
| – | act $= \text{SiLU}(\text{gate}) \odot \text{up}$ | *skipped* |
| 7 | down $= \text{act}\,W_{\text{down}}$ | $(M, d_{\text{ff}}) \times (d_{\text{ff}}, d)$ |
| – | Residual add | *skipped* |

Table 2: Llama model configurations used in our experiments. Size is the FP16 weight footprint of the seven projection matrices summed over all $L$ layers.

| | **1B** | **8B** |
|---|---|---|
| $d_{\text{model}}$ | 2,048 | 4,096 |
| $n_{\text{heads}}$ | 32 | 32 |
| $n_{kv\text{-heads}}$ | 8 | 8 |
| $d_{\text{head}}$ | 64 | 128 |
| $d_{\text{ff}}$ | 8,192 | 14,336 |
| $L$ | 16 | 32 |
| Size | 1.95 GB | 13.96 GB |

[1] The attention scores themselves ($QK^\top$ and $AV$) are also matrix multiplies, but at $M{=}1$ their inner dimension is the current sequence length $S$, which is small compared to $d_{\text{model}}$ and $d_{\text{ff}}$ for the regimes we study. Omitting them changes per-layer time by a small constant factor that is the same for AR, SD, and SSD and so does not affect the comparisons.

**Autoregressive decoding.** The simplest use of this compute graph is to model standard autoregressive decoding: starting from a single input token, we run the $L$-layer forward pass with $M{=}1$, producing one output token, and then repeat for $N$ iterations to simulate generating an $N$-token sequence. Each iteration is a sequential chain of $7L$ cuBLAS HGEMM launches with no inter-iteration parallelism, since the next token's input depends on the previous token's output. This serves as both our baseline (the AR throughput that SD and SSD must beat) and as the building block for the draft and target forward passes in the SD and SSD pipelines, which are the same compute graph invoked with different values of $M$.

### 3.3 Tensor Parallelism

We shard the target model across $T$ GPUs using 1D tensor parallelism in the style of Megatron-LM (Shoeybi et al., 2019). Tensor parallelism serves two goals. The first is to distribute the per-layer weight bandwidth demand across $T$ HBM subsystems, since single-token decode is bandwidth-bound rather than compute-bound. The second is to do so with minimal inter-GPU communication, since AllReduce latency directly competes with the compute it is meant to accelerate.

**Sharding strategy.** Each transformer layer in our compute graph consists of seven GEMMs: four attention projections ($W_Q, W_K, W_V, W_O$) and three FFN projections ($W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$). We split these into two groups:

- **Column-parallel:** $W_Q$, $W_K$, $W_V$, $W_{\text{gate}}$, $W_{\text{up}}$. Each GPU stores a $[d_{\text{model}}, d_{\text{out}}/T]$ slice of the weight matrix, sharding the *output* dimension. The input activation $X$ is replicated across all $T$ GPUs, and each GPU independently computes a $[M, d_{\text{out}}/T]$ slice of the output. No communication is required.

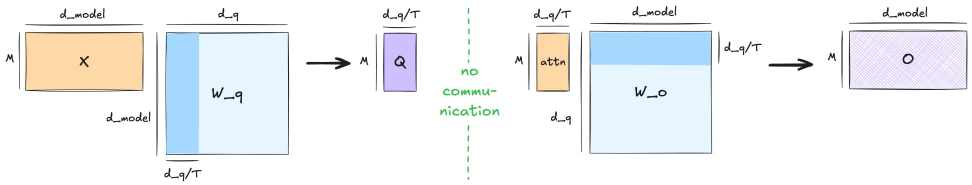
- **Row-parallel:** $W_O$, $W_{\text{down}}$. Each GPU stores a $[d_{\text{in}}/T, d_{\text{model}}]$ slice of the weight matrix, sharding the *input* dimension. Its input is the sharded output of the preceding column-parallel GEMM, so each GPU produces a $[M, d_{\text{model}}]$ *partial sum* of the true output. An AllReduce across the $T$ ranks recombines the partial sums.

This pairing—column-parallel feeding directly into row-parallel—is chosen specifically to eliminate a round of communication that a naive sharding would require. Consider what happens if both GEMMs in a pair were column-parallel instead. The first GEMM produces a sharded output along the inner dimension and the second GEMM needs that inner dimension *replicated* across ranks to compute correctly, forcing an AllGather between the two GEMMs. The same problem arises in reverse if both are row-parallel: the first GEMM produces a partial sum that must be reduced before the second can consume it, forcing an AllReduce between the two GEMMs. By pairing column-parallel with row-parallel, the sharded output of the first GEMM is exactly the sharded input the second GEMM expects, and the intermediate activation—$[M, d_q/T]$ between $W_Q$ and $W_O$, and $[M, d_{\text{ff}}/T]$ between $W_{\text{gate}}/W_{\text{up}}$ and $W_{\text{down}}$—stays entirely local. Communication is deferred to the boundary where it is mathematically unavoidable: the output of the row-parallel GEMM, which is a partial sum by construction. An illustration of this is shown in Figure 2.

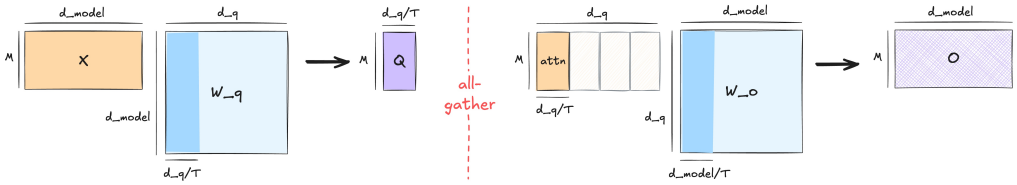
**Communication.** The result is exactly two AllReduce operations per layer (one after $W_O$, one after $W_{\text{down}}$) rather than the four a naively-sharded layer would require. We implement these with NCCL's `ncclAllReduce` in `ncclSum` mode over FP16.

Two properties of this pattern keep the communication cost bounded. First, each AllReduce transfers exactly $M \cdot d_{\text{model}}$ elements, *independent of* $T$—the message size does not grow with the number of GPUs, only the topology of the collective changes. Second, the elementwise operations a real transformer layer performs between the two row-parallel GEMMs—LayerNorm/RMSNorm, the residual addition, and the SwiGLU activation between the gate/up projections and the down projection—operate purely on the (already-reduced) $d_{\text{model}}$ activation or on the sharded intermediate, and so require no additional communication. We omit these elementwise operations from our compute graph (Section 3.2), but the sharding choice is compatible with them at zero communication cost.

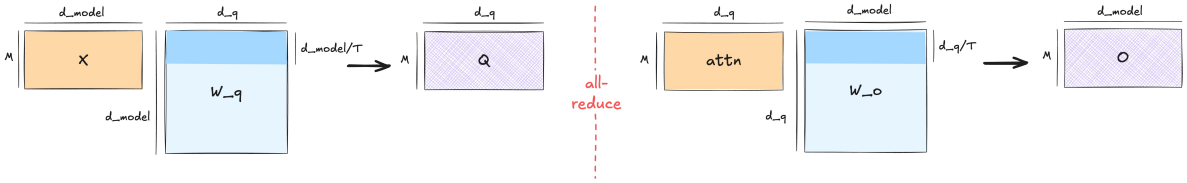
**Why this limits scaling.** For single-token decode ($M = 1$), each GEMM degenerates to a vector–matrix multiply whose runtime is dominated by reading the weight slice from HBM. Sharding the weights $T$-ways reduces the per-GPU read by a factor of $T$, but the AllReduce payload of $d_{\text{model}}$




(a) *One column-parallel, one row–parallel.* The sharded output of the first GEMM matches the sharded input of the second GEMM, requiring no communication between the the first and second GEMM.




(b) *Both column-parallel.* The second GEMM needs each GPU's slice of attn as input, forcing an AllGather between the first and second GEMM.




(c) *Both row-parallel.* The first GEMM produces a partial sum that must be combined with each GPU's partial sum before it can be used as input to the second GEMM, forcing an AllReduce between the first and second GEMM.

Figure 2: Comparison of tensor parallelism weight sharding strategies. By pairing column-parallel with row-parallel, no intermediate communication is required. All methods produce a partial sum after the second GEMM, so they all need an AllReduce at the end.

FP16 elements is small enough to be latency-bound rather than bandwidth-bound on NVLink. Once the per-GPU compute time falls below the all-reduce latency, adding more GPUs stops helping. We characterize where this crossover occurs for our target model dimensions in Section 4.4.

**Implementation.** Each rank runs in its own host thread with a dedicated `cublasHandle_t`, `ncclComm_t`, and `cudaStream_t`. Weights are allocated once at startup directly into their sharded shapes—no full-size weights are ever materialized—so the per-GPU memory footprint scales as $1/T$ of the unsharded model. The model registry enforces that $d_{\text{q}}$, $d_{\text{kv}}$, and $d_{\text{ff}}$ are all divisible by $T$. For grouped-query attention this further requires $T \leq n_{\text{kv\_heads}}$, which holds for all configurations we evaluate (Llama-3.2-1B and Llama-3.1-8B both have $n_{\text{kv\_heads}} = 8$).

### 3.4 SD Pipeline

We implement speculative decoding directly on top of the compute graph of Section 3.2. The draft and target are two separate instantiations of the same graph at different model sizes, and an SD round is a fixed sequence of forward passes over them followed by an acceptance step.

**Round structure.** Let $K$ be the speculation length. One SD round consists of (i) $K$ sequential draft forward passes at $M{=}1$, producing $K$ candidate tokens, followed by (ii) one target forward pass at $M{=}K$, which scores all $K$ candidates in a single batched call, followed by (iii) acceptance, which determines how many of the $K$ candidates are committed and adds one bonus token. We loop until the cumulative committed-token count reaches a configurable horizon $N$, and we report wall-clock time, total tokens, and total rounds for the iteration.

**Synthetic Bernoulli acceptance.** Because no real logits are produced, we cannot evaluate the modified rejection-sampling rule of Leviathan et al. (2023) or Chen et al. (2023) on the actual draft and target distributions. Instead, we sample acceptance from a `std::bernoulli_distribution` at a configurable rate $\alpha$ on the host. After the target's

verification pass returns, we draw $K$ independent Bernoulli($\alpha$) outcomes and accept the longest prefix of consecutive successes. The round commits that prefix plus one bonus token, exactly as in the real algorithm. The expected number of tokens per round is $\sum_{i=0}^{K} \alpha^i = (1-\alpha^{K+1})/(1-\alpha)$, which is the same closed form Leviathan et al. (2023) derive for the true acceptance rate. Treating $\alpha$ as an independent knob lets us sweep it in Section 4 without coupling it to a particular pair of real models, and it isolates exactly the quantity that determines SD's speedup over AR.

**Single-GPU pipeline.** The single-GPU configuration is the direct translation of the round structure above. One cuBLAS handle drives both the draft and target forward passes on a single CUDA stream, and the host generates Bernoulli outcomes between rounds. Because the draft and target share a stream, all GPU work serializes naturally.

**Tensor-parallel pipeline.** When the target is sharded across $T$ GPUs (Section 3.3), the round structure is the same but the coordination is more involved. The draft model fits on a single GPU and runs only on rank 0. The target runs on all $T$ ranks in lockstep. We spawn one host thread per rank, each owning a `GPUContext` (cuBLAS handle, NCCL communicator, CUDA stream) bound to its device. Rank 0 additionally owns a *second* cuBLAS handle and CUDA stream dedicated to the draft model, so that draft GEMMs do not interleave with the target's TP-sharded GEMMs and AllReduces on the shared stream. In the SD pipeline this separation is purely for cleanliness, but it is the same mechanism SSD will later use to overlap draft and target execution (Section 3.5).

A single round proceeds in three phases separated by a `pthread_barrier`:

1. **Draft phase.** Rank 0 runs $K$ sequential draft forward passes on the draft stream and synchronizes that stream. Other ranks wait at the barrier.

2. **Verify phase.** All ranks call the function `forward_model_tp` with $M{=}K$, which issues the seven sharded GEMMs and two AllReduces per layer described in Section 3.3, and then synchronize the TP stream.

3. **Accept phase.** Rank 0 (on host) draws $K$ Bernoulli outcomes, updates the shared committed-token counter, and signals round-completion through a `std::atomic<bool>`. All ranks read the flag at the next barrier and either re-enter the loop or exit.

The barriers are needed because cuBLAS and NCCL calls only synchronize work on a stream, not across host threads, and a rank that races ahead would corrupt the TP collective for the next round. Timing is recorded on rank 0's TP stream with CUDA events so that the reported wall-clock figure reflects the full round including the host-side acceptance step. As in the single-GPU case, weights and activations are allocated once at startup in their sharded shapes. Only the control state—the round counter, the atomic flag, and the random generator—lives on the host.

**Component timing.** In addition to end-to-end measurement, the benchmark separately times $T_{\text{draft step}}$ (one draft forward at $M{=}1$), $T_{\text{target step}}$ (one target forward at $M{=}1$), and $T_{\text{target verify}}$ (one target forward at $M{=}K$). These are measured in isolated micro-benchmarks and they let us decompose observed speedups in Section 4 into the components that the SD speedup formula

$$\text{speedup}_{\text{AR}\to\text{SD}} = \frac{E_{\text{SD}}}{1 + T_{\text{SD}}}$$

(where $E_{\text{SD}} = (1-\alpha^{K+1})/(1-\alpha)$ and $T_{\text{SD}} = T_{\text{draft}}/T_{\text{target}}$) predicts.

### 3.5 SSD Pipeline

Speculative decoding leaves a measurable inefficiency on the table: in every round the draft and target alternate, but never overlap. During speculation, much of the GPU sits idle. Speculative speculative decoding (SSD) (Kumar et al., 2026) eliminates this idle time by running the draft *concurrently* with verification, on dedicated hardware, and pre-speculating for likely verification outcomes so that when the target's outcome arrives the next round's speculation is already waiting.

We implement SSD on top of the same compute-graph and tensor-parallel primitives as our SD pipeline (Sections 3.2, 3.3, 3.4). The contribution at this layer is not a new GEMM kernel but the *coordination* of three asynchronous activities: target verification on $T$ GPUs, draft pre-speculation on a $(T{+}1)$th GPU, and inter-GPU communication that exchanges outcomes and speculations between the two. This subsection describes how we lay those

activities out across hardware, what the per-round protocol looks like, how we use multiple CUDA streams to expose the overlap that gives SSD its speedup, and which modeling choices we made to keep the implementation tractable in our GEMM-only setting.

**Hardware layout.** An SSD run uses $T+1$ GPUs. Ranks $0, \ldots, T-1$ form the tensor-parallel target and use the same column/row-parallel sharding and two-AllReduce-per-layer schedule described in Section 3.3. Rank $T$ is the dedicated *draft* GPU. It holds an unsharded copy of the (small) draft model. This is the first point at which SSD diverges from SD: in our SD pipeline the draft co-locates with target rank 0 and the two share hardware, but for SSD the draft must run *simultaneously* with verification, which forces it onto its own device. Putting the draft on its own GPU also means the draft's HBM bandwidth, kernel launch queues, and stream resources are entirely uncontended—the draft loop never competes with the target's TP collectives for either compute or NVLink.

**Two NCCL communicators.** A single NCCL communicator cannot serve both the TP AllReduces and the draft–target point-to-point messages, because the TP communicator's topology is sized for $T$ ranks and excludes the draft GPU, and because mixing collective and send/recv traffic on one communicator would force them to serialize. We therefore initialize *two* NCCL communicators at startup:

- `tp_comm`: size $T$, ranks $\{0, \ldots, T-1\}$, used for the per-layer AllReduces inside `forward_model_tp`;
- `dt_comm`: size 2, with rank 0 on target rank 0 and rank 1 on the draft, used for sending verification outcomes from the target to the draft and returning speculations the other way.

Only target rank 0 and the draft join `dt_comm`, the other target ranks operate purely through `tp_comm`. This separation is what makes the outcome/speculation exchange a two-party operation that does not stall the rest of the TP target.

**Per-round protocol.** We implement the speculator/verifier loop of Kumar et al. (2026), specialized to a single batch and adapted to the GEMM-only setting. After a one-time priming step in which the draft sends an initial speculation produced by a just-in-time draft pass, every steady-state round on the *target* side performs three actions:

1. Rank 0 issues `ncclRecv` on `dt_comm` for the $K$ speculation tokens posted by the draft. Ranks $1, \ldots, T-1$ wait at a host-side `pthread_barrier`.
2. All $T$ ranks call `forward_model_tp` with $M{=}K$, issuing the seven sharded GEMMs and two AllReduces per layer.
3. Rank 0 draws synthetic Bernoulli($\alpha$) outcomes (Section 3.4), determines the accepted prefix length and a done flag, and sends the resulting 3-int outcome packet `[acc_len, bonus_id, done]` on `dt_comm` via `ncclSend`.

The barrier is required because cuBLAS and NCCL synchronize work within a stream but not across host threads, and a rank that raced into the next round would corrupt the TP AllReduce for that round.

The *draft* side runs a structurally different loop. After priming, each iteration is:

1. *Pre-speculate.* Issue $K$ draft forward passes at $M{=}F$ on the compute stream, asynchronously. These represent speculations for $F$ candidate verification outcomes.
2. *Receive outcome.* Issue `ncclRecv` for the target's outcome packet on the *communication* stream, then block on that stream and copy the outcome to host.
3. *Decide hit/miss.* If a synthetic Bernoulli($p_{\text{hit}}$) draw indicates a cache hit, the pre-speculated batch already contains a usable speculation and we proceed directly. If it is a miss, we run an additional $K$ *just-in-time* draft passes at $M{=}1$—the same fallback SD would have used—and increment the miss counter.
4. *Send speculation.* Issue `ncclSend` on the communication stream to deliver the next speculation to target rank 0, and synchronize.

The entire loop terminates when the target's outcome packet carries `done = 1`, signaling that the cumulative committed-token count has reached the configurable horizon $N$. On termination the draft drains any in-flight pre-speculation work via `cudaStreamSynchronize` so that the next benchmark iteration starts from a clean stream.

**Two-stream draft for compute/communication overlap.** The performance argument for SSD rests on hiding draft latency *inside* the target's verification window. Achieving this in practice requires that the draft's pre-speculation GEMMs and its `ncclRecv` of the target's outcome run on independent streams. We therefore give the draft GPU two CUDA streams: a `compute_stream` (bound to the cuBLAS handle, on which all `hgemm` launches enqueue) and a `comm_stream`, used exclusively for `dt_comm` traffic. The two streams are populated in the order shown above: pre-speculation enqueues onto `compute_stream` *first*, then `ncclRecv` enqueues onto `comm_stream`, and only `comm_stream` is synchronized to retrieve the outcome. The CUDA driver is then free to execute the GEMM launches on the SMs in parallel with the NCCL recv on the copy/NIC engine, exactly the overlap the SSD timeline requires.

Target rank 0, by contrast, uses a *single* stream for both its TP collectives and its `dt_comm` send/recv operations. The target's per-round work is strictly sequential—receive speculation, run `forward_model_tp`, send outcome—so issuing those operations on a shared stream produces the correct ordering for free and avoids the scheduler indeterminacy of a second stream.

**Fan-out.** The Saguaro algorithm of Kumar et al. (2026) pre-speculates a *set* of $F$ candidate continuations, indexed by predicted verification outcomes, and looks up the actual outcome in a speculation cache. In a real implementation, $F$ controls both how much draft compute is spent per round and how high the cache hit rate $p_{\text{hit}}$ can climb. The two are coupled by the draft's sampling distribution.

In our compute-graph setting we do not run a real model, so there is nothing to look up: $F$ and $p_{\text{hit}}$ become independent configuration knobs. We model fan-out by issuing the $K$ pre-speculation passes at *batch size* $M{=}F$ rather than $M{=}1$, which reproduces the GEMM shapes and HBM traffic the real draft would experience when speculating for $F$ candidate outcomes in parallel within a single forward pass. We model the cache lookup by sampling a single Bernoulli($p_{\text{hit}}$) per round on the draft host. This decoupling is intentional: it lets us sweep the two quantities independently in Section 4 and isolate their contributions to end-to-end speedup, where in a real system they would always co-vary.

**Per-rank thread structure.** Each of the $T+1$ ranks runs in its own host thread, spawned via `std::thread`. A thread allocates its own `cublasHandle_t`, its CUDA streams, the relevant subset of NCCL communicators (`tp_comm` for target ranks, `dt_comm` for rank 0 and the draft, both for rank 0), and either the TP-sharded target weights or the unsharded draft weights. The draft additionally allocates two activation buffers of differing batch dimensions: one sized for $M{=}1$ (used for just-in-time fallback) and one sized for $M{=}F$ (used for pre-speculation). All allocations happen once at startup. Per-iteration round termination is signalled through a `std::atomic<bool>` read at the bottom of each round. Only target rank 0 writes it, so no heavier synchronization primitive is needed.

## 4 Results

### 4.1 Experimental Setup

**Hardware and software.** All experiments run on PSC Bridges-2 V100-32GB nodes, with up to 4 NVIDIA Tesla V100 GPUs used per allocation connected via NVLink 2.0. The benchmark binary is compiled with `nvcc` (CUDA 12.x) against cuBLAS for the GEMMs and NCCL for the inter-GPU collectives and point-to-point messages.

**Models.** We evaluate on two configurations from the Llama family: Llama-3.2-1B as a draft model and small-target stand-in, and Llama-3.1-8B as the primary target model. Their dimensions are listed in Table 2 (Section 3.2).

**Inputs.** Because the benchmark uses the GEMM-only compute graph of Section 3.2, there are no real prompts or token sequences. Each end-to-end run generates $N{=}128$ "tokens" (i.e., terminates the round loop once the cumulative committed-token count reaches 128).

**Measurement methodology.** Every reported timing is the mean over 30 timed iterations preceded by 10 warmup iterations, recorded with `cudaEventRecord` and `cudaEventElapsedTime` on the relevant CUDA stream (target rank 0's stream for SD and SSD, the lone rank's stream for AR). Component-level timings ($T_{\text{draft step}}$, $T_{\text{target verify}}$, $T_{\text{prespec}}$) come from isolated microbenchmarks that allocate fresh weights and activations and time only the sections of interest. The NCCL AllReduce and send/recv microbenchmark numbers we use to attribute TP overhead are produced by a standalone harness that runs alongside the main benchmark on the same hardware.

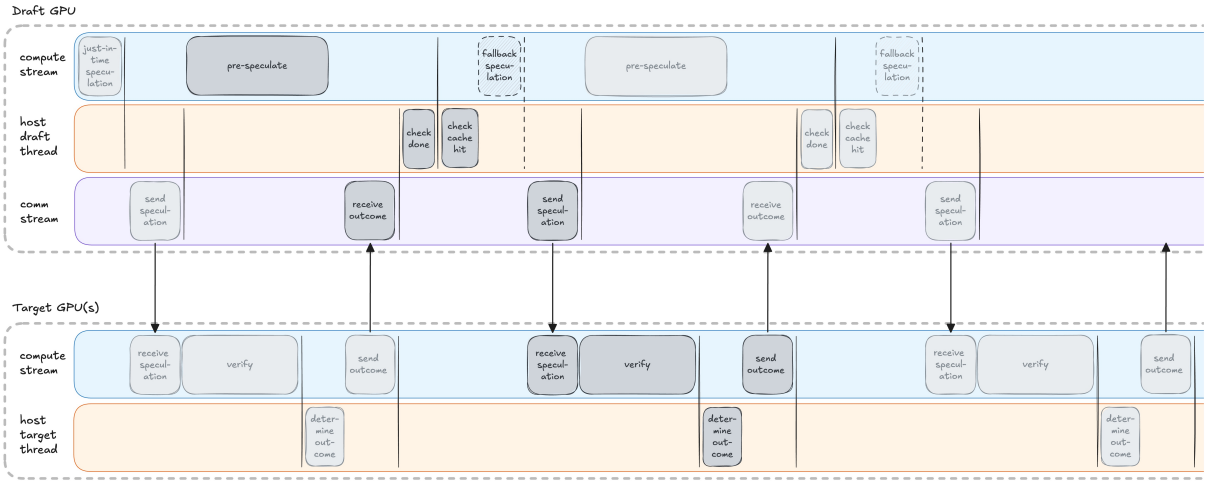



Figure 3: Per-round SSD execution across the draft GPU (top) and the target GPU(s) (bottom), separated into compute streams, host threads, and the draft's communication stream. Arrows indicate `dt_comm` send/recv messages between target rank 0 and the draft. Vertical lines indicate `cudaStreamSynchronize` points. The draft's pre-speculation of the next round runs concurrently with target verification. The first round (leftmost) is primed with a just-in-time speculation since no pre-speculation is available yet. Steps colored in the same shade of gray indicate that they are part of the same round.

**Metrics.** The headline metric is end-to-end *throughput* in tokens per second, defined as $N$ divided by the wall-clock time of one $N$-token decode. Where useful we also report per-round wall-clock time (round time = $N$/throughput $\times$ tokens-per-round), the per-round token yield (which is 1 for AR and approximately $E_{\text{SD}} = (1-\alpha^{K+1})/(1-\alpha)$ for SD/SSD), and the empirical miss rate for SSD. Speedups are always reported as throughput ratios.

**Configurations and baselines.** Throughout, AR is the baseline: a single-stream loop of $N$ forward passes at $M{=}1$ on the target model, with no draft and no concurrency. SD and SSD are compared against AR (for absolute speedup) and against each other (for the SSD-over-SD relative gap that motivates the project). The SSD pipeline uses $T{+}1$ GPUs ($T$ for the tensor-parallel target, one for the dedicated draft). The SD pipeline uses $T$ GPUs total with the draft co-located on rank 0. Default values when not swept are $\alpha{=}0.9$, $K{=}4$, $F{=}4$, $p_{\text{hit}}{=}0.85$.

**Goals against the proposal.** Our proposal set three quantitative targets. SD-over-AR was projected at 3–4$\times$ at $\alpha{=}0.9$, $K{=}5$. We measure 4.08$\times$ at $\alpha{=}1.0$, $K{=}8$ and 3.79$\times$ at $\alpha{=}0.9$, $K{=}8$, in line with this target. SSD-over-SD was projected at 15–25% at $p_{\text{hit}}{=}0.85$. We measure 32–53% across $\alpha \in [0.5, 1.0]$ at the equivalent operating point, comfortably exceeding the target.

### 4.2 AR vs. SD vs. SSD Throughput

The first question is whether the implementation actually delivers the speedups its scheduling structure predicts. We sweep the two parameters that the SD speedup formula is most sensitive to—the synthetic acceptance rate $\alpha \in \{0.5, 0.7, 0.8, 0.9, 1.0\}$ and the speculation length $K \in \{1, 2, 4, 6, 8\}$—with the Llama-3.1-8B target and Llama-3.2-1B draft, $T{=}1$, $N{=}128$, and (for SSD) the fan-out and cache hit rate fixed at $F{=}4$ and $p_{\text{hit}}{=}0.85$. Each point is the mean over 30 timed iterations after 10 warmup iterations. AR is independent of $\alpha$ and $K$ by construction and runs once.

There are three key observations from Figure 4.

**All three methods converge as $K \to 1$.** At $K{=}1$ the pre-speculation window collapses to a single $M{=}1$ draft pass ($\sim$3.2 ms in our component timing), so SSD has almost nothing to overlap with the target's verification. SSD still beats SD by roughly 12% across $\alpha$ at $K{=}1$, because even a single short draft pass can be hidden under the verify, but the gap is narrow—most of the round time is the verification itself, which both methods pay identically. AR sits at 49.1 tok/s, set entirely by the target step time of 20.4 ms.

**SD's optimum tracks $\alpha$; SSD's does not.** As $\alpha$ grows, SD's optimal $K$ moves right—$K{=}2$ at $\alpha{=}0.5$, $K{=}4$ at $\alpha{=}0.8$, $K{=}8$ at $\alpha \geq 0.9$—

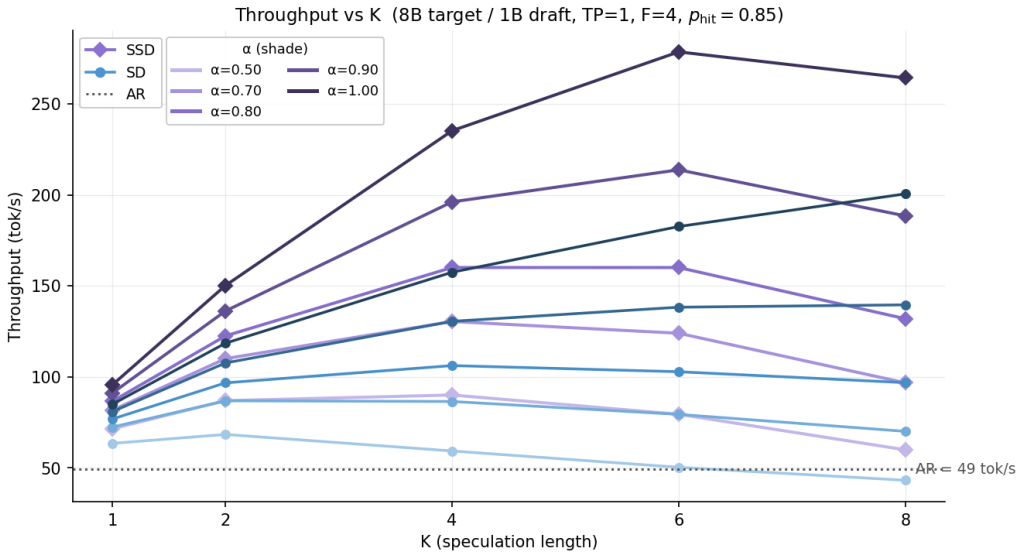



Figure 4: End-to-end throughput (tokens/second) versus speculation length $K$, for AR, SD, and SSD. Curve shade encodes $\alpha$. AR (dotted) is independent of $\alpha$ and $K$. SSD's optimum sits at a smaller $K$ than SD's at every $\alpha$, and SSD regresses past $K{=}6$: this is the verification-window ceiling discussed in the text.

which is exactly what the closed form $E_{\text{SD}} = (1{-}\alpha^{K+1})/(1{-}\alpha)$ predicts: the marginal benefit of an additional speculated token is $\alpha^{K+1}$, so high acceptance rates justify spending more draft time per round. SSD's optimum sits at $K{=}4$ for $\alpha \leq 0.8$ and $K{=}6$ for $\alpha \geq 0.9$, and—more notably—SSD's curve *rolls over* at $K{=}8$ for every $\alpha$. The optimal $K$ for SSD is smaller than for SD, and capped from above by something other than the SD speedup formula.

**The rollover follows directly from the round-time budget.** Once we account for the actual stream layout described in Section 3.5, this is mechanical. A steady-state SSD round on the draft GPU enqueues $K$ pre-speculation passes at $M{=}F$ on the compute stream and an `ncclRecv` on the communication stream, and the round latency is bounded below by $\max(T_{\text{verify}}, T_{\text{prespec}})$ plus the `dt_comm` exchange. Component timing (Table 3) shows that $T_{\text{prespec}}$ grows linearly in $K$ while $T_{\text{verify}}$ stays essentially flat: at $M \leq 8$ the target forward is bandwidth-bound, so verify time is set by the cost of streaming weights from HBM and is nearly independent of $M$. The crossover between $T_{\text{verify}}$ and $T_{\text{prespec}}$ falls between $K{=}4$ and $K{=}6$, and from there on every additional unit of $K$ adds round latency rather than amortizing over more accepted tokens.

This is the central qualitative finding of the sweep: **SSD's optimal $K$ is set by the verification window, not by the SD speedup curve.** The two methods are solving different optimization problems. SD trades draft work for accepted tokens and tunes $K$ against $\alpha$, SSD already has the draft work hidden under verify and tunes $K$ against the size of the verification window. The sweet spot for SSD is the largest $K$ that keeps the round verify-bound.

Table 3: SSD round-time components ($\alpha{=}1.0$, $F{=}4$, $p_{\text{hit}}{=}0.85$). The round is bounded below by $\max(T_{\text{verify}}, T_{\text{prespec}})$. The crossover between $K{=}4$ and $K{=}6$ explains why SSD's optimum is at $K{=}6$ and why SSD regresses at $K{=}8$.

| $K$ | $T_{\text{verify}}$ (ms) | $T_{\text{prespec}}$ (ms) | Bound by |
|---|---|---|---|
| 1 | 20.3 | 3.5 | verify |
| 2 | 18.8 | 7.0 | verify |
| 4 | 18.8 | 14.0 | verify |
| 6 | 18.9 | 21.0 | *prespec* |
| 8 | 19.0 | 28.1 | *prespec* |

**Speedup over AR.** Figure 5 reports the speedup over AR at each method's best $K$. SSD reaches $5.67\times$ at $\alpha{=}1.0$ ($K{=}6$) versus SD's $4.08\times$ ($K{=}8$), and is consistently faster than the best SD configuration by 32–53% across the swept range. The relative gap is largest in the middle of the $\alpha$ range, peaking at 51% at $\alpha{=}0.8$, and narrowing slightly at $\alpha{=}1.0$ because SD's $E_{\text{SD}}$ keeps growing in $\alpha$ even after SSD's verify window has saturated.

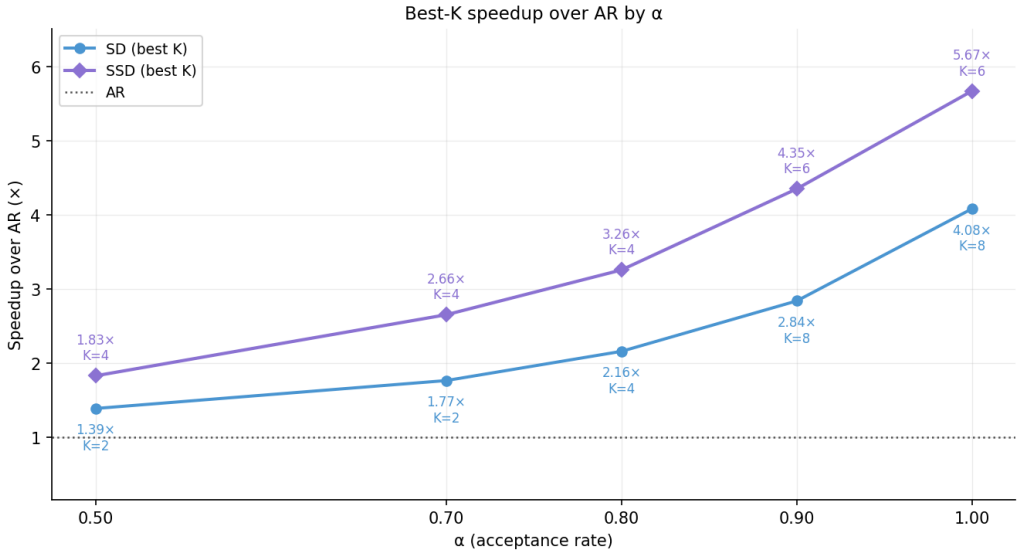



Figure 5: Speedup over AR at each method's best $K$, as a function of $\alpha$. SSD's best $K$ is 4 for $\alpha \leq 0.8$ and 6 for $\alpha \geq 0.9$. SD's best $K$ drifts from 2 at $\alpha$=0.5 to 8 at $\alpha \geq 0.9$. SSD is faster than the best SD configuration by 32–53% across the swept range, with the largest relative gap at $\alpha \in [0.7, 0.9]$.

**Calibration with the SD speedup formula.** The SD points in Figure 5 can be checked directly against the speedup formula of Section 3.4. Component timing at $K$=8 gives $T_{\text{draft total}} = 25.88$ ms and $T_{\text{target}} = 20.32$ ms, so $T_{\text{SD}} = 1.27$ and the formula predicts $E_{\text{SD}}/(1 + T_{\text{SD}}) = 9/(1 + 1.27) = 3.96\times$ at $\alpha$=1.0. The measured value is $4.08\times$, within 3% of theory. SSD does not admit such a clean closed form because its denominator becomes $\max(1, T_{\text{prespec}}/T_{\text{target}}) + \varepsilon_{\text{NCCL}}$ rather than $1 + T_{\text{SD}}$, and its numerator is reduced on miss rounds—we measure these effects directly in the cache-hit-rate and fan-out sweeps in Section 4.3.

## 4.3 Impact of Cache Hit Rate and Fan-Out

The throughput sweep of Section 4.2 fixed the two SSD-specific knobs at $p_{\text{hit}}$=0.85 and $F$=4. The SSD speedup expression in Section 4.2 identified two quantities that determine SSD's advantage over SD: the per-round denominator $\max(T_{\text{verify}}, T_{\text{prespec}})$, which is governed by $F$ (since $T_{\text{prespec}}$ scales with the pre-speculation batch size), and the per-round token yield in the numerator, which is reduced on miss rounds and so depends on $p_{\text{hit}}$. We measure each separately by fixing $\alpha$=0.9, $K$=4 and sweeping one knob at a time: $p_{\text{hit}}$ probes the numerator, $F$ probes the denominator.

### 4.3.1 Cache Hit Rate

Figure 6 reports SSD throughput relative to SD as $p_{\text{hit}}$ ranges from 0.3 to 1.0. Two things stand out.

**SSD beats SD across the entire swept range.** Even at $p_{\text{hit}}$=0.3—a regime where two of every three rounds fall back to a just-in-time draft—SSD is $1.12\times$ faster than SD. The crossover where SSD ceases to beat SD lies below the lowest $p_{\text{hit}}$ we measured. This is more robust than we expected. The intuition is that on a miss, SSD pays roughly the same per-round cost as SD (verify followed by $K$ JIT draft passes), so the miss path is approximately neutral in expectation. The hit path is strictly faster because pre-speculation runs concurrently with verification rather than sequentially. As long as some fraction of rounds hit, SSD is ahead, and the marginal benefit per percentage point of $p_{\text{hit}}$ is roughly the SD draft-to-target ratio.

**The relationship is approximately linear in $p_{\text{hit}}$.** Going from $p_{\text{hit}}$=0.3 to $p_{\text{hit}}$=1.0, the speedup grows from $1.12\times$ to $1.64\times$—a fairly steady $\sim 0.07\times$ per 0.1 of hit rate. This is consistent with a simple model where the per-round time is $T_{\text{verify}} + (1 - p_{\text{hit}}) \cdot K \cdot T_{\text{draft}}$ on the miss path (since the hit path is verify-bound), and tokens per round are independent of $p_{\text{hit}}$ at fixed $\alpha$. The empirical miss rate matches the declared $p_{\text{hit}}$ closely (within 0.01 across all points), so the synthetic-Bernoulli model in Section 3.5 is doing what we wanted—decoupling $p_{\text{hit}}$ from the actual draft sampling dis-

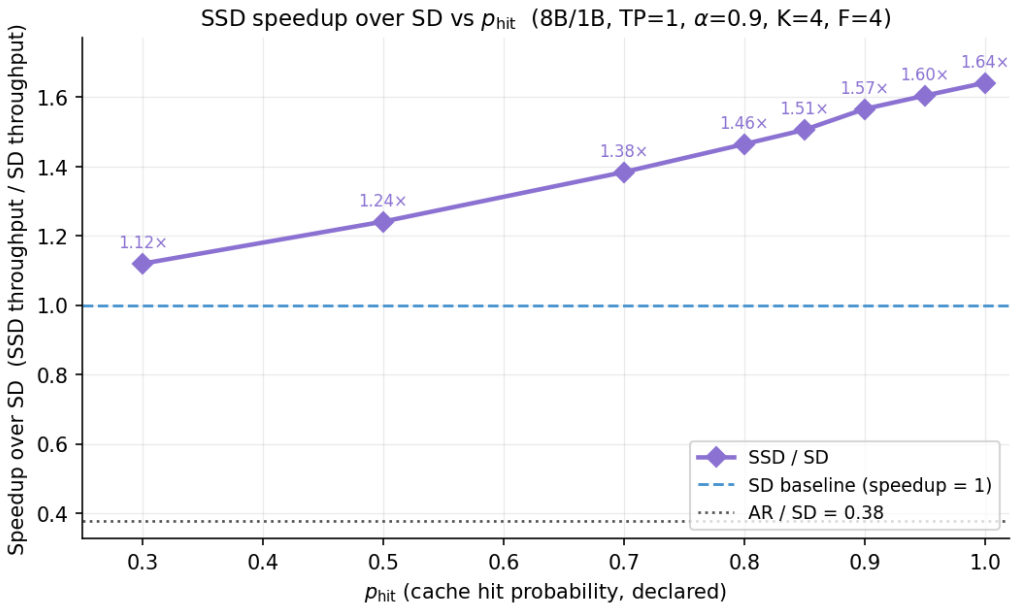



Figure 6: SSD speedup over the best SD configuration as a function of declared cache hit probability $p_{\text{hit}}$, at $\alpha$=0.9, $K$=4, $F$=4. SSD beats SD across the entire swept range, including at $p_{\text{hit}}$=0.3.

tribution lets us read the sensitivity off cleanly.

In a real Saguaro-style implementation (Kumar et al., 2026), $p_{\text{hit}}$ is set by the quality of the draft model's outcome predictor. In our compute-graph setting it is a configuration knob. The takeaway from this sweep is that the design is robust to mediocre prediction: SSD's advantage degrades gracefully as $p_{\text{hit}}$ falls and only loses to SD at hit rates below the floor of our sweep.

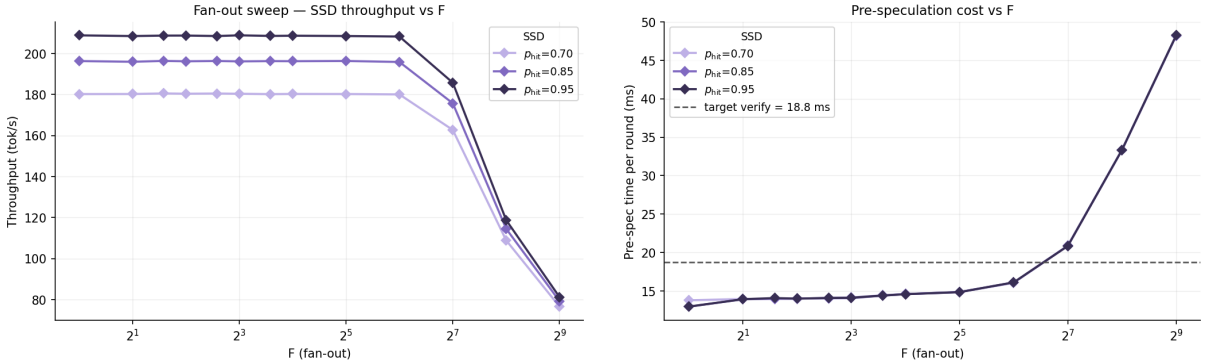
#### 4.3.2 Fan-Out

Fan-out $F$ controls how many candidate verification outcomes the draft pre-speculates for in parallel. As described in Section 3.5, we model fan-out by issuing the $K$ pre-speculation passes at batch size $M{=}F$ rather than $M{=}1$, which reproduces the GEMM shapes and HBM traffic the real draft would experience. We sweep $F$ over $\{1, 2, 3, 4, 6, 8, 12, 16, 32, 64, 128, 256, 512\}$ at three hit rates (0.7, 0.85, 0.95) with $\alpha$=0.9 and $K$=4 fixed.

**Fan-out is effectively free up to $F$=64.** Throughput is constant within measurement noise for $F \leq 64$ (Figure 7a). At all three hit rates, every $F$ from 1 to 64 delivers within 0.5% of the same throughput. This is the central result of the sweep: in our regime the draft GPU has substantial headroom, and there is no penalty for pre-speculating wide. A real Saguaro implementation can spend that headroom on predicting more candidate outcomes and driving $p_{\text{hit}}$ up, with no cost on the round time as long as $F$ stays under the threshold where pre-speculation breaks out of the verify window.

**The throughput cliff at $F \geq 128$ is the same verify-window mechanism as the $K$-rollover.** Beyond $F$=64 throughput collapses—to ∼165–185 tok/s at $F$=128, ∼110–120 at $F$=256, ∼76–82 at $F$=512. Figure 7b explains why: pre-speculation time stays flat and small until $F \approx 64$, then turns up sharply and crosses the target verify line (18.8 ms) between $F$=64 (16.1 ms) and $F$=128 (20.9 ms). Once $T_{\text{prespec}} > T_{\text{verify}}$, the round becomes pre-spec-bound, the overlap that gives SSD its speedup is gone, and every doubling of $F$ adds round latency proportionally. This is structurally identical to the $K$=8 rollover in Section 4.2: both are crossings of the same $\max(T_{\text{verify}}, T_{\text{prespec}})$ threshold, just along different axes.

**The flat region is set by HBM, not $p_{\text{hit}}$.** The three $p_{\text{hit}}$ curves in Figure 7b sit on top of each other: pre-speculation cost depends on $F$ and $K$ but not on $p_{\text{hit}}$ (which only affects how often the JIT fallback runs, not how long pre-speculation itself takes). The fact that pre-spec time is ∼14 ms across all small $F$ reflects that single-token decode is bandwidth-bound, as discussed in Section 3.3: the $K$ pre-spec passes at small $M$ are dominated by streaming the draft weights from HBM, and in-




(a) SSD throughput vs. $F$. Throughput is essentially flat for $F \leq 64$ and collapses thereafter.

(b) Pre-speculation cost per round vs. $F$, with target verify time as the dashed reference. The crossover at $F \approx 64$–$128$ explains the throughput collapse in (a).

Figure 7: Fan-out sweep at $\alpha$=0.9, $K$=4, for three values of $p_{\text{hit}}$. The three $p_{\text{hit}}$ curves in (b) are essentially indistinguishable.

creasing $M$ from 1 to 64 adds compute but does not add HBM traffic, so the wall-clock cost is unchanged. The cliff at $F \geq 128$ corresponds to the regime where the GEMMs become compute-bound and runtime starts scaling with $F$. The exact crossover depends on the draft model size and the GPU's compute-to-bandwidth ratio, but the qualitative shape—a flat shelf followed by a cliff—will hold for any draft/target/hardware combination.

### 4.4 Per-Round Latency Breakdown and TP Scaling

The throughput differences between AR, SD, and SSD in Section 4.2 can be read off the round-time structure of each method. We make this explicit in two ways: a single-configuration decomposition that shows where each round's wall-clock time goes (Figure 8), and a TP scaling sweep that decomposes how those round times change as the target is sharded across more GPUs (Figure 9). The TP decomposition is anchored to NCCL AllReduce microbenchmarks which gives us a quantitative attribution for where TP scaling falls short of ideal.

**The three round shapes.** Figure 8 shows the canonical case at TP=1. AR's round is one target step at 20.2 ms. SD's round stacks two segments—$K \cdot T_{\text{draft}}$=12.9 ms of sequential drafting on top of $T_{\text{verify}}$=19.0 ms of verification—for a total of 31.9 ms. The draft and verify run on the same CUDA stream and serialize naturally: nothing happens during the draft except draft, and nothing happens during the verify except verify. SSD's round occupies 21.1 ms in total, with $\max(T_{\text{verify}}, T_{\text{prespec}})$=18.8 ms of useful work and the dashed line at 13.9 ms marking the pre-speculation that ran concurrently inside the verify window. The thin band at the top of the SSD bar is 2.3 ms of host-side coordination overhead (the `pthread_barrier_wait` calls, the host-to-device copy of the outcome packet, the cache-hit Bernoulli draw, and the `cudaStreamSynchronizes`). The two `dt_comm` send/recv operations in this overhead account for only $\sim$25 $\mu$s per round in the NCCL sendrecv microbenchmark—less than 2% of the residual—so the coordination cost we see is dominated by host-side synchronization, not by the interconnect. Each SD/SSD round delivers $\sim$4.1 tokens (the expected yield $E_{\text{SD}} = (1-\alpha^{K+1})/(1-\alpha)$ at $\alpha$=0.9, $K$=4) versus AR's one, so the taller bars do correspondingly more work—throughput is round time per token, not round time alone.

SD has no analogous residual at TP=1 because the round runs entirely on a single CUDA stream with no cross-thread coordination: its component microbenchmarks ($T_{\text{draft}}$ and $T_{\text{verify}}$) already include their own kernel launch overhead, and putting them in sequence in the end-to-end loop adds nothing measurable. We omit the SD residual from the figure for this reason.

**The asymmetry that makes SSD win.** The visual story is in the relative bar heights. SD's round is 11.7 ms taller than AR's because draft is added to verify. SSD's round is 0.9 ms shorter than AR's because draft is hidden under verify and the small coordination tax does not fully cancel the savings. Said differently: at TP=1, going from AR to SD costs you 58% more round time and gains you 4.05 tokens per round (vs. AR's 1), for a net 1.62×

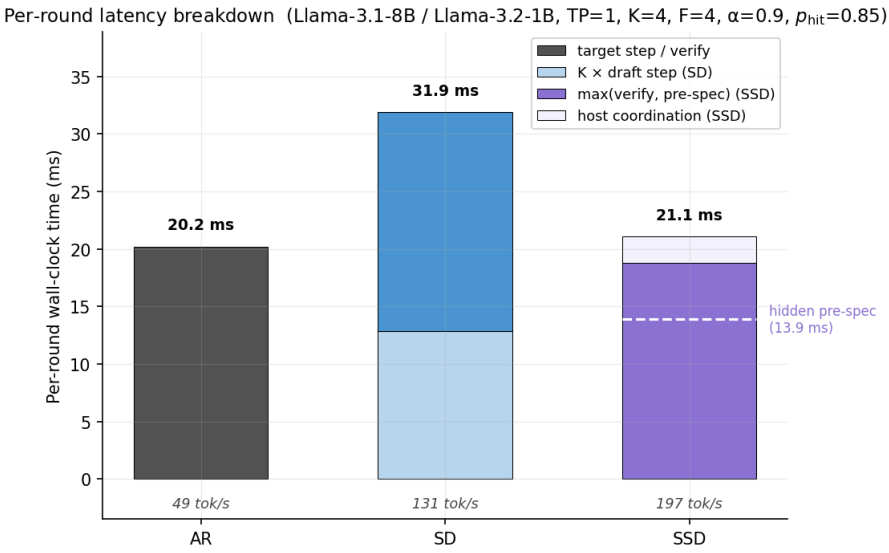



Figure 8: Per-round wall-clock time for AR, SD, and SSD at the canonical operating point ($\alpha$=0.9, $K$=4, $F$=4, $p_{\text{hit}}$=0.85, TP=1, Llama-3.1-8B target / Llama-3.2-1B draft). Throughput shown below each bar. The dashed line inside the SSD bar marks the hidden pre-speculation that runs concurrently with verification.

throughput. Going from AR to SSD costs you 4% more round time and gains you the same 4.05 tokens per round, for a net 4.01× throughput. The figure makes the mechanism visible: SSD wins because the round it pays for is barely longer than AR's, while delivering SD's per-round token yield.

**TP scaling on the target is sublinear, and the microbenchmark says why.** Figure 9 sweeps TP from 1 to 4 on the 8B target with the same $K$=4, $F$=4, $\alpha$=0.9 configuration. AR's round time falls from 20.2 ms to 13.2 ms to 9.7 ms—a 2.1× speedup at TP=4 rather than the 4× ideal. The verify portion of every bar is split into three components: the bottom (solid) is $1/T$ scaling of the TP=1 measurement, the middle (hatched) is 64 AllReduces (2 per layer × 32 layers, per Section 3.3) at the per-call latency the NCCL microbenchmark reports for that $T$ and payload, and the top (light) is the residual gap. At TP=2 the hatched AllReduce segment is 0.9 ms (64 calls × 14.1 $\mu$s for a 32,768-byte payload at np=2), and the residual launch/dispatch gap is another 1.0 ms, for a total of 1.9 ms above ideal. At TP=4 the AllReduce segment grows to 1.5 ms (64 × 22.7 $\mu$s at np=4) and the residual to 1.2 ms, for a total of 2.7 ms above ideal. The microbenchmarks predict roughly half of the gap from ideal scaling at both TP values. The rest is launch and dispatch overhead from the cuBLAS calls, which become a larger fraction of the round as the per-rank GEMM shrinks.

This is the crossover Section 3.3 predicted in the abstract: once per-GPU compute time falls below the AllReduce latency, additional GPUs stop helping. We can now name where it happens for our target: at TP=4 on V100, the 1.5 ms of AllReduce time is already 20% of AR's 7.5 ms ideal compute, and the launch/dispatch residual adds another 20%. A TP=8 sweep would push compute below 4 ms while AllReduce latency continues to grow, and we would expect TP scaling to flatten out entirely. We were unable to run a TP=8 sweep due to compute resources availability.

**SSD inherits SD's TP scaling on the verify side.** Comparing SD and SSD bars within each TP group, SSD's verify portion is the same size as SD's at every TP—both methods run the same TP-sharded `forward_model_tp` kernel, so they pay the same AllReduce cost and the same dispatch overhead. The difference is that SD adds a ∼13 ms draft segment on top of the verify, and SSD adds only its ∼2–3 ms host coordination. The result is that SSD's lead over SD widens as TP grows: SSD's round is 33% shorter than SD's at TP=1, and 33% shorter at TP=2 (16.3 vs. 24.3 ms). The relative gap is preserved because both methods see the same verify shrinkage, while SD's draft segment stays fixed—draft runs on a single GPU and is not affected by the target's tensor parallelism.

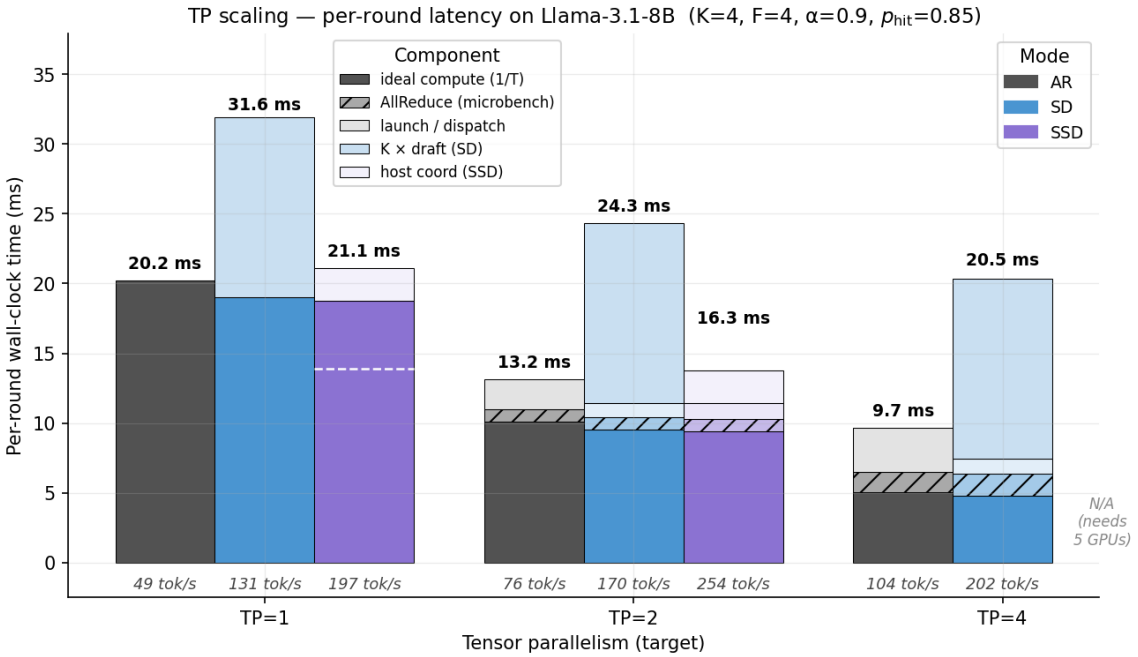



Figure 9: Per-round latency on Llama-3.1-8B as the target is sharded across $T \in \{1, 2, 4\}$ GPUs. Each bar's verify portion is decomposed into ideal $1/T$ compute, AllReduce overhead from the NCCL microbenchmark (hatched), and residual launch/dispatch overhead (light). SSD at TP=4 requires 5 GPUs and was not run.

### 4.5 Roofline Analysis

We measure how close our forward-pass implementation gets to the cuBLAS GEMM lower bound, and characterize where TP scaling falls short of its $1/T$ ideal.

**Constructing the roofline.** We microbenchmark each of the four unique GEMM shapes that appear in a layer—qkv (the fused Q/K/V projection), `attn_out`, `ffn_up` (which has the same shape as `ffn_gate`), and `ffn_down`—for both Llama-3.2-1B and Llama-3.1-8B at batch dimensions $M \in \{1, 2, 4, 8, 16, 32\}$. The microbenchmark is a standalone harness that does nothing but issue `cublasHgemm` calls in a tight loop with cache flushes, on a single V100. This isolates the per-GEMM cost from any overhead our forward-pass code might add (CUDA stream coordination, NCCL setup, the per-iteration scratch buffer reuse).

Let $T_{\text{layer}}(M) = T_{\text{qkv}}(M) + T_{\text{attn\_out}}(M) + 2\,T_{\text{ffn\_up}}(M) + T_{\text{ffn\_down}}(M)$ be the predicted time for one layer at batch dimension $M$. The predicted per-pass latency at tensor parallelism $T$ is then

$$\hat{T}(M, T) = \frac{L \cdot T_{\text{layer}}(M)}{T},$$

where $L$ is the layer count (16 for 1B, 32 for 8B), the four $T_{\text{kind}}(M)$ values come from the GEMM microbenchmarks, and the $1/T$ scaling reflects that tensor parallelism shards the per-layer GEMM work across $T$ ranks (Section 3.3). This is a strict lower bound on the per-pass latency our implementation can achieve in two senses: it omits all overhead from how our code orchestrates the GEMMs (kernel launch latency, host dispatch, CUDA stream coordination), and the $1/T$ factor assumes perfect linear TP scaling with no AllReduce cost.

**Measured points.** Against this roofline we plot (Figure 10) two kinds of measurements from our implementation: AR's target step (a single forward at $M{=}1$) and SD's target verify (a forward at $M{=}K$, with $K \in \{4, 8\}$). We omit SD's $M{=}1$ verify because it duplicates the AR step up to measurement noise, and we omit SSD entirely because its target side runs the same `forward_model_tp` as SD with the same shapes, yielding identical points. Each measured point is the mean of 30 timed iterations after warmup.

**The TP=1 column shows the implementation is GEMM-tight.** At TP=1, every measured point falls within 3–6% of the prediction: the AR ratios are 1.05 (1B) and 1.06 (8B), and the SD-verify ratios are 1.05 (1B) and 1.03 (8B). On the 8B model these ratios correspond to absolute gaps of 1.2 ms (AR) and 0.6 ms (SD verify K=4). With $L{=}32$

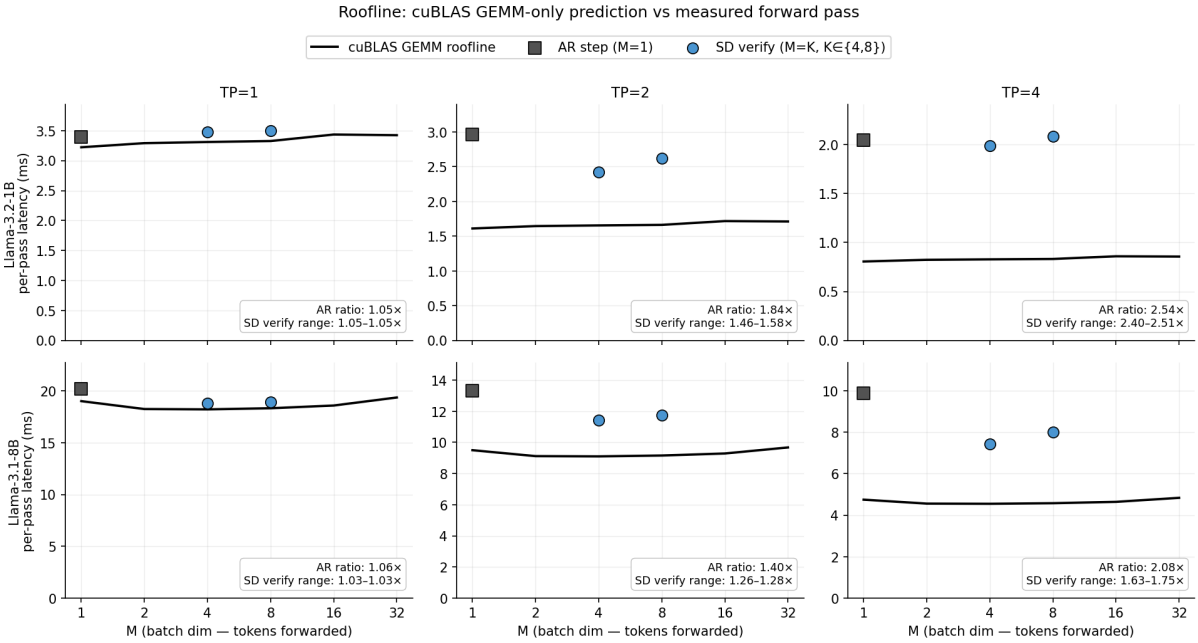



Figure 10: Predicted per-pass latency from the cuBLAS GEMM microbenchmark (solid line) versus measured forward passes from the SD pipeline (markers), for Llama-3.2-1B (top row) and Llama-3.1-8B (bottom row), at TP $\in \{1, 2, 4\}$. The $1/T$ scaling assumes perfect tensor parallelism and is an unattainable lower bound; the gap between markers and line is everything our implementation pays beyond the raw GEMMs themselves. Corner boxes report the AR (M=1) and SD-verify (M=K) measured-to-predicted ratios for each panel. All measured points lie in the bandwidth-bound regime $M \leq 32$, where $\hat{T}(M)$ grows weakly with $M$.

layers and 7 GEMM launches per layer, our forward pass issues 224 host-side cuBLAS dispatches. If the per-call cuBLAS launch latency is in the standard few-microsecond range, the dispatch overhead alone would be on the order of 1 ms, which accounts for most of the gap from prediction. We take this as confirmation that our implementation introduces no significant overhead beyond what cuBLAS itself charges for issuing the GEMMs—there is no orchestration tax from the way we structure the forward pass at TP=1.

**The TP > 1 columns measure where TP scaling falls short.** As $T$ grows, the gap from line to points opens up. On the 8B model the AR ratio rises from 1.06 at TP=1 to 1.40 at TP=2 to 2.08 at TP=4. The measured forward pass at TP=4 takes twice as long as the perfect-$1/T$ scaling of the GEMM microbench would predict. The SD-verify ratios rise less steeply ($1.03 \rightarrow 1.26 \rightarrow 1.63$–$1.75$), and the 1B model shows the same pattern more sharply (AR ratio reaches 2.54 at TP=4). Two effects compound: AllReduce, which the $1/T$ prediction omits, and the diminishing per-rank GEMM size, which makes kernel launch overhead a larger fraction of the total. Section 4.4 attributed these costs explicitly using the NCCL AllReduce microbenchmark, finding that AllReduce alone accounts for roughly half of the gap from ideal scaling at TP=2 and TP=4 on the 8B target. The roofline view here is consistent with that decomposition: the gap exists, it grows with $T$, and it is the same gap visualized as a fraction of predicted time rather than as a stacked component.

**Bandwidth-bound regime.** The roofline lines themselves are nearly flat across $M \in \{1, \ldots, 32\}$. This is the bandwidth-bound shelf discussed in Section 3.3: at small $M$, each GEMM is dominated by streaming the weight matrix from HBM, and the work the GPU does on those weights scales with $M$ but the bytes read do not. The compute-bound regime where per-pass latency would scale linearly with $M$ lies above $M$=32 for both models on V100, well past any SSD operating point we benchmark ($K \leq 8$, $F \leq 64$). The flat shelf is what makes SSD's verification window roughly K-independent in Section 4.2 and what makes the fan-out budget of $F \approx 64$ in Section 4.3 possible—both findings are downstream consequences of the same bandwidth-bound roofline shape.

### 4.6 Throughput–Latency Pareto Frontier

When choosing a serving configuration, one cares about two quantities: per-sequence latency and total throughput. We sweep the throughput–latency frontier on the 8B target across the eight configurations formed by combining method (AR vs. SD vs. SSD) with tensor parallelism ($T \in \{1, 2, 4\}$), with $\alpha$=0.9, $K$=4, $F$=4, and $p_{\text{hit}}$=0.85 fixed. Figure 11 shows the throughput–latency frontier.

**Reading the plot.** Per-token latency is end-to-end wall-clock time divided by tokens generated, computed from the same end-to-end runs as Section 4.4. Each method's three configurations connect with a trendline, and the upper-right corner is the dual optimum: lowest latency *and* highest throughput. AR's frontier sits in the lower-left (49 tok/s at 20.2 ms latency for TP=1, climbing to 104 tok/s at 9.7 ms for TP=4) and is the slowest method on both axes at every TP. SD pulls the frontier up and to the right (131 $\rightarrow$ 170 $\rightarrow$ 202 tok/s as TP grows). SSD pulls it further still: at TP=1, SSD already matches SD at TP=4 in throughput (197 vs. 202 tok/s) while running on a quarter of the target GPUs. At TP=2, SSD reaches 254 tok/s at 3.94 ms per token—the highest throughput *and* the lowest latency of any configuration we measured.

**Why SSD/TP=2 wins on both axes simultaneously.** The two-axis dominance comes from two compounding effects, both established in earlier subsections. On the latency axis, TP shrinks $T_{\text{verify}}$ (from 19 ms at TP=1 to 11.4 ms at TP=2 in Section 4.4), and SSD's pre-speculation hides under that shrunken verify so the round time tracks $T_{\text{verify}}$ rather than $T_{\text{verify}} + K \cdot T_{\text{draft}}$. On the throughput axis, each round still produces $\sim$4 tokens (set by $\alpha$ and $K$, independent of TP), so a shorter round translates directly into more rounds per second. The TP knob and the SSD coordination strategy are independent levers (Section 4.4, last paragraph), so applying both compounds: the round both produces more tokens than AR's and takes less wall-clock time than AR's at the same TP.

**What changes the frontier shape.** The frontier we report is for $\alpha$=0.9, $K$=4, $F$=4, $p_{\text{hit}}$=0.85, which is a moderately favourable operating point. Two of the underlying parameters can shift the frontier in ways our earlier sweeps quantify. Lower $\alpha$ (rejection-prone draft) compresses SD's curve toward AR (Section 4.2: SD's speedup over AR collapses from 4.08$\times$ at $\alpha$=1.0 to 1.39$\times$ at $\alpha$=0.5) but compresses SSD's curve less because the verify-window mechanism is largely $\alpha$-independent (SSD speedup over AR drops from 5.67$\times$ to 1.83$\times$, a 32% smaller relative compression). Lower $p_{\text{hit}}$ (worse outcome predictor) compresses SSD toward SD but, as Section 4.3 showed, SSD continues to beat SD for $p_{\text{hit}}$ as low as 0.3. So the qualitative shape of the frontier—SSD on top, SD in the middle, AR on the bottom—is robust across the parameter regime we explored. What changes is the magnitude of the gap.

## References


Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *Preprint*, arXiv:2302.01318.

Tanishq Kumar, Tri Dao, and Avner May. 2026. Speculative speculative decoding. *Preprint*, arXiv:2603.03251.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. *Preprint*, arXiv:2211.17192.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*.


## A Work Distribution

Table 4 lists the work performed by each partner.

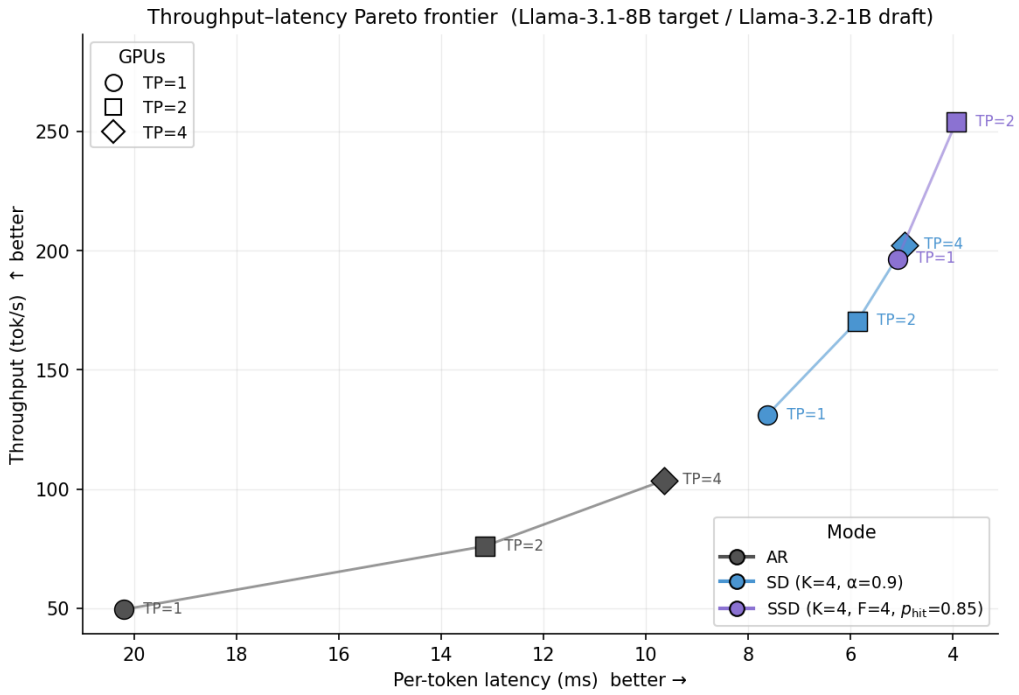



Figure 11: Throughput–latency frontier for Llama-3.1-8B target / Llama-3.2-1B draft, with $\alpha$=0.9, $K$=4, $F$=4, $p_{\text{hit}}$=0.85. The x-axis is per-token latency (lower is better; reversed so that "better" moves right) and the y-axis is throughput (higher is better). Marker color indicates method, marker shape indicates GPU count. SSD with TP=2 (3 GPUs total: 2 target + 1 draft) Pareto-dominates every other configuration we measured.

| **Tasks** | **Marcus** | **William** |
|---|---|---|
| **Overall** | 50% | 50% |
| Compute Graph | 90% | 10% |
| AR Pipeline | 90% | 10% |
| SD Single-GPU Pipeline | 10% | 90% |
| Tensor Parallelism | 25% | 75% |
| SD TP Pipeline | 10% | 90% |
| SSD Pipeline | 90% | 10% |
| NCCL Communicators | 50% | 50% |
| Stream Coordination | 50% | 50% |
| GHC Testing | 40% | 60% |
| PSC Testing | 50% | 50% |
| GEMM Microbenchmarks | 90% | 10% |
| NCCL Microbenchmarks | 90% | 10% |
| $\alpha/K$ Sweep | 10% | 90% |
| $p_{\text{hit}}/F$ Sweep | 10% | 90% |
| TP Scaling Sweep | 10% | 90% |
| Roofline Analysis | 90% | 10% |
| Pareto Analysis | 90% | 10% |
| Figure Creation | 60% | 40% |
| Project Proposal | 50% | 50% |
| Milestone Report | 50% | 50% |
| Final Report | 50% | 50% |
| Poster Creation | 10% | 90% |

Table 4: Distribution of work by task.